

RT-normalization spline fits: root cause and a minimum-occupancy binning guard

2026-08-11

Contents

Summary	1
How it surfaced	2
Root cause 1: a column that can never be filled	2
Why it usually does not throw	3
Root cause 2: no minimum bin occupancy	3
Proposed design	4
Minimum occupancy with neighbour merging	4
In <code>getQuantSplines</code>	4
Choosing the two constants	5
Required downstream change	5
Making the solve robust	6
Alternatives considered	6
Verification	6
Not addressed here	7

Summary

Two separate defects sit behind the `SingularException` that failed the `SearchDIA_yeast` snoop target.

1. **A structural off-by-one in the spline design matrix.** `UniformSpline` allocates `n_knots + 3 = 10` basis columns, but the tenth can never hold a non-zero value. Every RT-normalization fit in Pioneer is therefore rank deficient by at least one column – including the healthy files with 8,700 PSMs. It throws only in the narrow case where the system happens to be square, because that is the only case Julia routes through `lu`.
2. **No minimum occupancy in the RT binning.** `N_sample = min(N, nprecs)` gives a 14-PSM file fourteen bins of one PSM each, so the “median per bin” is just the raw value and the fitted correction is noise.

The first explains the crash exactly. The second is the statistical defect, and is the one that silently damages results.

Recommendation: fix the binning by minimum occupancy (merge undersized bins into their neighbour; if a single merged bin is still undersized, skip normalization for that file), and make the solve robust so a

rank-deficient system can never throw. Treat the design-matrix off-by-one as a separate, carefully validated change.

How it surfaced

A local `create_app` build of `feat/integ-minwidth5-order3-portable` failed in `SearchDIA_yeast`:

```
LinearAlgebra.SingularException(10)
  MaxLFQSearch.jl:195  summarize_results!
  normalizeQuant.jl:137  normalizeQuant
  normalizeQuant.jl:55  getQuantSplines
  uniformBasisCubicSpline.jl:227  UniformSpline
```

Bisected to a single change:

test	result
branch as-is	fails, <code>SingularException(10)</code>
3rd-order WH smoothing disabled	still fails, <code>SingularException(2)</code>
<code>feature/portable-packaging-phase1</code> (no integration changes)	passes
only <code>apex_padded +/-2</code> reverted to <code>+/-1</code>	passes

The trigger is the minimum 5-scan integration width. It did not introduce a defect in `normalizeQuant`; it perturbed quantification enough to move one file onto the one bin count that crashes.

Root cause 1: a column that can never be filled

`_build_numeric_design_matrix` allocates `n_cols = length(knots) + 3`. For each data point it finds `knot_idx` and writes four basis values into columns `knot_idx : knot_idx+3`.

With `n_knots = 7`, knot indices run 1..7, so columns 1..10 are all *written*. But counting non-zeros over 100 evenly spaced points:

column	1	2	3	4	5	6	7	8	9	10
non-zero rows	17	33	50	65	66	65	50	33	17	0

Column 10 is reached only by the single point at $t = \max(t)$, where $u = 0$, and that basis function is zero at $u = 0$. So it is written, always with the value zero.

The consequence is not data-dependent:

bins K	8	9	10	12	20	50	100
rank of X	8	9	9	9	9	9	9
empty columns	1	1	1	1	1	1	1

Rank 9 of 10 at every bin count, including 100. Every quant-normalization spline Pioneer has ever fitted is rank deficient.

Why it usually does not throw

Julia's `\` dispatches on shape. Non-square goes to QR, which returns a solution for a rank-deficient system without complaint. Square goes to lu, which raises `SingularException(k)` on a zero pivot at column `k`. Reproduced directly:

```
K=9   (X is 9x10, non-square -> qr)  -> OK
K=10  (X is 10x10, SQUARE    -> lu)  -> SingularException(10)
K=11  (X is 11x10, non-square -> qr)  -> OK
```

The (10) in the production stack trace is the index of the structurally empty column. The crash needs `K == n_coeffs` exactly – a one-in-many coincidence that the integration change happened to land on.

This also explains the `SingularException(2)` seen with 3rd-order smoothing disabled: a square system whose *second* column had no support, which happens when the bin centres are clustered away from that knot span.

Root cause 2: no minimum bin occupancy

Measured on the yeast fixture by instrumenting the fit:

With the minimum-width change (fails):

file	PSMs	N_sample	bin_size	distinct median iRTs	iRT span
lowsignal.arrow	14	14	1	13	0.851
rep1.arrow	8,705	100	87	100	20.536
rep2.arrow	8,710	100	87	100	20.297

With it reverted (passes):

file	PSMs	N_sample	bin_size	distinct median iRTs	iRT span
lowsignal.arrow	18	18	1	17	1.134
rep1.arrow	8,634	100	86	100	19.448
rep2.arrow	8,636	100	86	100	19.827

`bin_size = 1`. The binning has collapsed: `median_rts` is not a set of bin medians, it is the sorted per-PSM iRTs. The averaging that makes this estimator meaningful on `rep1/rep2` (87 PSMs per bin) is entirely absent.

That file's spline then enters `getQuantCorrections` weighted equally with files of 8,700 PSMs:

```
for (key, spline) in pairs(quant_splines)
    median_quant[j, i] = spline(rt_grid[i])    # every file contributes equally
    j += 1
end
median_quant = reshape(median(median_quant, dims = 1), (N,))
```

and is evaluated across `rt_grid`, which spans the union of all files' RT ranges (~20 units) versus the 0.851 units it was fitted on. `UniformSpline` clamps outside its range, so the contribution is an extrapolated constant derived from noise – both to the cross-file median every other file is corrected against, and to every precursor in that file via `applyNormalization!`.

Proposed design

Minimum occupancy with neighbour merging

Replace “fixed number of bins” with “bins of at least *m* PSMs”:

```
"""
    _occupancy_bins(n, min_occupancy, max_bins) -> Vector{UnitRange{Int}}
```

Contiguous bin ranges over *n* RT-sorted PSMs, each holding at least *min_occupancy* rows, and at most *max_bins* bins.

Undersized bins are merged into their neighbour rather than kept, so the median per bin is always taken over at least *min_occupancy* values. Returns a single range covering everything when *n* cannot support two bins, and an empty vector when it cannot support even one -- the caller treats that as "do not normalize this file".

```
"""
function _occupancy_bins(n::Int, min_occupancy::Int, max_bins::Int)
    n >= min_occupancy || return UnitRange{Int}[]
    k = min(max_bins, n / min_occupancy |> floor |> Int) # bins we can fill
    k <= 1 && return [1:n]
    base, rem = divrem(n, k) # spread the remainder; sizes differ by <= 1
    bins = Vector{UnitRange{Int}}(undef, k)
    start = 1
    for i in 1:k
        len = base + (i <= rem ? 1 : 0)
        bins[i] = start:(start + len - 1)
        start += len
    end
    return bins
end
```

k = *n* / *min_occupancy* is the merge, expressed directly: rather than forming *N_{sample}* bins and merging the short ones pairwise, compute how many bins the data can fill and lay them out evenly. The result is the same and there is no iteration to get wrong.

In getQuantSplines

```
bins = _occupancy_bins(npreds, min_bin_occupancy, N)
```

```
if length(bins) < min_bins_for_spline
    @user_warn "Skipping quant normalization for $(basename(fpath)): " *
        "$npreds PSMs support only $(length(bins)) RT bin(s) at >= " *
        "$min_bin_occupancy PSMs each; a $(spline_n_knots + 3)-coefficient " *
        "spline needs at least $min_bins_for_spline. Abundances are left " *
        "uncorrected and excluded from the cross-file median."
    continue
end
```

```
median_quant = [log2(median(@view psms[b, quant_col_name])) for b in bins]
median_rts    = [median(@view psms[b, :irt_obs]) for b in bins]
splinefit = UniformSpline(median_quant, median_rts, 3, spline_n_knots)
insert!(quant_splines, fpath, splinefit)
```

Choosing the two constants

`min_bin_occupancy` is a statistical choice. The median of `m` samples has standard error about $1.25 \text{ sigma} / \sqrt{m}$, so `m = 1` gives the raw value and `m = 100` roughly 0.125 sigma .

Chosen: `m = 100`.

Two consequences worth stating plainly, because neither is neutral:

- **Healthy files rebin.** `rep1` has 8,705 PSMs. Today it gets 100 bins of 87; at `m = 100` it gets 87 bins of 100. The spline is fitted to different points, so quantification shifts slightly for every file, not only the degenerate ones. Measured on the `ecoli` fixture: identifications are unchanged (3,427 precursors / 1,731 protein groups for Altimeter, 3,206 / 1,637 for Prosit, before and after), while the median normalized abundance moves +1.2% (Altimeter) and -0.2% (Prosit).
- **Small files are no longer normalized.** With `m = 100` and a floor of 11 bins, a file needs about 1,100 PSMs before it is corrected at all. That is the intended behaviour – below that there is not enough signal to estimate an RT-dependent correction – but it is a larger exclusion than `m = 10` would give, and it is worth knowing that a sparse run now passes through uncorrected rather than being corrected badly.

`min_bins_for_spline` must be **strictly greater than `n_coeffs`**, not equal to it. Requiring `>= n_coeffs` would permit `K == 10`, which is precisely the square case that throws. Until the design matrix is fixed this should be `n_coeffs + 1 = 11` at minimum, and more realistically `2 * n_coeffs` so the fit is overdetermined rather than barely determined.

This corrects an error in the first draft of this note, which proposed `length(median_rts) >= n_coeffs` and would have permitted the crashing case.

With `m = 100` and `min_bins = 11`, `lowsignal` (14 PSMs) yields zero bins, falls below the threshold, and is skipped – the correct outcome. The emitted warning is:

```
Skipping quant normalization for lowsignal.arrow: 14 PSMs support only 0 RT
bin(s) at >= 100 PSMs each, and a 10-coefficient spline needs at least 11.
Abundances are left uncorrected and excluded from the cross-file median.
```

There is a second, quieter benefit to `m = 100`. The crash needs exactly `n_coeffs = 10` bins, which at `m = 100` means a file of 1,000-1,099 PSMs – and every such file yields 10 bins, below the floor of 11, so it is skipped. No PSM count produces a bin count that is both at or above the floor and equal to `n_coeffs`, which makes the square-system case unreachable rather than merely unlikely. There is a unit test asserting exactly that.

Required downstream change

`getQuantCorrections` iterates `pairs(quant_splines)` and needs no change; a skipped file simply does not participate.

`applyNormalization!` does:

```
correction_spline = corrections[fpath]           # KeyError for a skipped file
```

becomes

```
correction_spline = get(corrections, fpath, nothing)
if correction_spline === nothing
    psms[!, norm_quant_col] = Float64.(psms[!, quant_col]) # pass through unchanged
    writeArrow(fpath, psms)
    continue
end
```

Emitting the `_normalized` column unchanged rather than omitting it keeps the Arrow schema uniform, which downstream readers assume.

Making the solve robust

Independently of the binning, the fit should not be able to throw on a rank-deficient system. The cheapest change is to stop letting shape decide the algorithm – a pivoted QR handles both cases and returns a least-norm solution where a column has no support:

```
c = qr(basis.X, ColumnNorm()) \ sorted_u
```

This is behaviour-preserving for the overdetermined case all healthy files already take, and removes the square-system crash class entirely. It should go in regardless of which binning policy is adopted.

Alternatives considered

Fix the design matrix off-by-one. The honest fix for root cause 1: the tenth column contributes nothing to fitted values anywhere on the evaluation domain (the spline clamps to `[first, last]`), so dropping it is equivalent in exact arithmetic and makes the system full rank. But `_coeffs_to_spline` and `_build_piecewise_matrix` both assume `n_knots + 3` coefficients, so this touches spline machinery used elsewhere – including RT alignment and quad transmission. Worth doing, but as its own change with its own validation, not folded into a quantification fix.

Penalized fit (`UniformSplinePenalized`, `lambda > 0`). Makes $H = X'X + \text{lambda} * D$ positive definite so the solve cannot fail. Removes the crash but keeps the statistical problem: it still produces a correction for a file with no information to justify one, and still admits that file to the cross-file median.

Adaptive spline `n_knots` per file. Lower the knot count until the fit is identifiable. More faithful in principle, but files would then be normalized against a median assembled from splines of differing flexibility.

Deduplicate identical median RTs. Addresses one narrow way rank is lost. The failing file has 13 distinct values out of 14, so this would remove one point and change nothing.

Verification

Status as implemented:

1. **SearchDIA_yeast** passes with the minimum-width change in place, emitting the skip warning for `lowsignal.arrow`. Done.
2. **Ecoli searches produce identical identifications.** Altimeter 3,427 precursors / 1,731 protein groups and Prosit 3,206 / 1,637, before and after. Median normalized abundance moves +1.2% and -0.2% respectively – expected, since `m = 100` rebins the healthy files too. Done.
3. **721 unit tests** in `test/UnitTests/test_quant_bin_occupancy.jl`: bin invariants (occupancy floor, sizes within one, exact tiling, contiguity), occupancy-limited counts, degenerate collapse, defensive parameters, and the threshold's relationship to the crash. Done.
4. **The crash is pinned by a test**, not just avoided: the suite asserts `UniformSpline` throws `SingularException` at exactly `n_coeffs` points and succeeds at one either side. If a future change removes the empty column, that test fails loudly rather than leaving a threshold nobody can justify.

Not done: a check that the `rep1/rep2` spline coefficients themselves move only as much as the rebinning explains. The identification counts being unchanged is good evidence but not proof.

Not addressed here

- **Whether the minimum 5-scan integration width is correct.** It is the trigger, not the defect, but it reduced `lowsignal` from 18 identified PSMs to 14. Widening an integration window should not usually cost identifications, and that deserves its own look.
- **Whether `N = 100` bins is right** for large files now that occupancy is the binding constraint.
- **The other `UniformSpline` callers.** The empty-column defect is in the shared basis construction, so RT alignment and quad transmission fits are rank deficient in the same way. None is known to have thrown, because none has hit the square case – but the fits are equally over-parameterised.